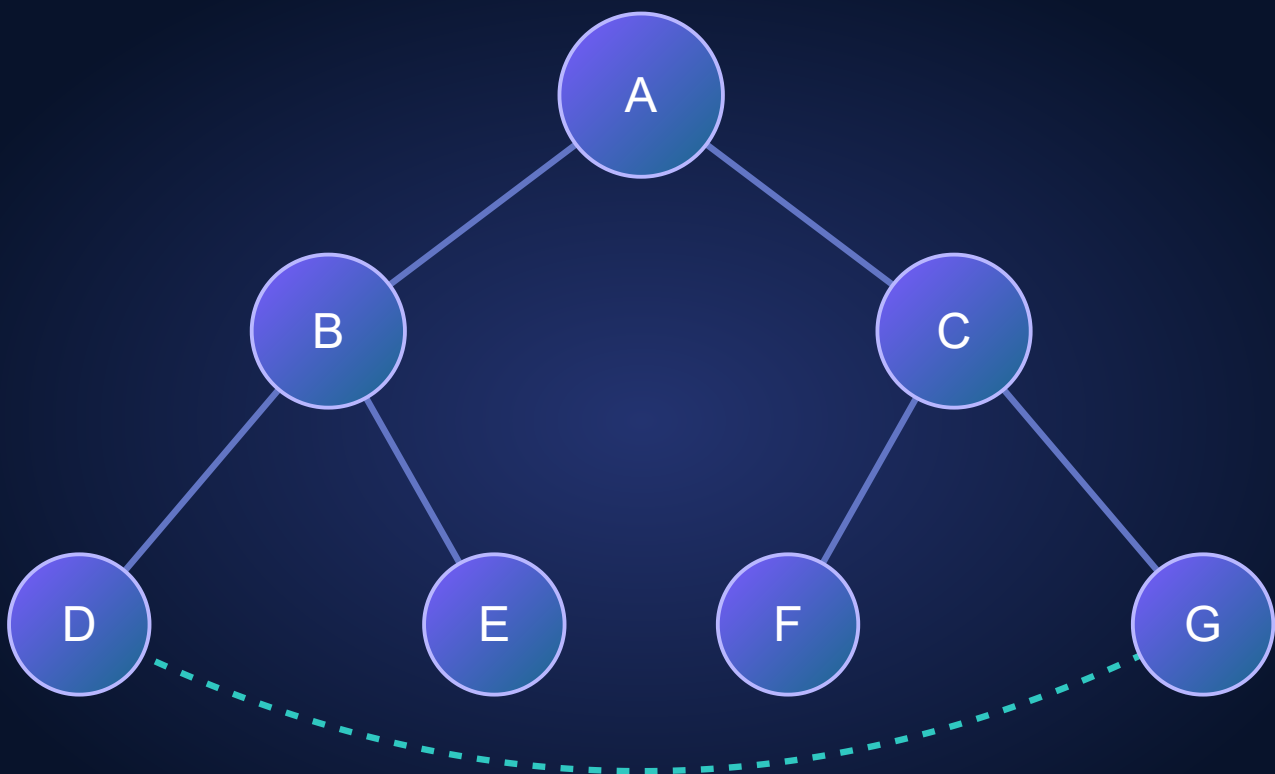


PROGRAMMING · INTERMEDIATE

Data Structures & Algorithms

See the algorithm. Understand every step.



TRACE · REASON · COMPARE

A visual, practice-first path through data structures, algorithms and interview-grade problem solving.



12-16 weeks



5 modules



Professional learning

[Explore the learning experience →](#)

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

Learn by manipulating live arrays, linked nodes, trees and graphs; step through algorithms; inspect state and complexity; and solve progressively harder coding challenges.



Choose the right data structure for a constraint



Trace and implement common algorithms



Explain time and space complexity



Solve interview-grade problems with tested code



Who this is for

- C and C++ learners
- Software engineering students
- Developers preparing for technical interviews



Your starting point

- Programming fundamentals in C, C++, Java, Python or JavaScript

Your progression at a glance

FROM CONCEPT TO EVIDENCE



Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 Complexity, Arrays and Strings

Reason about cost and contiguous structures.

- Big O with live traces
- Array memory visualizer
- Two pointers and sliding windows
- Searching and sorting

2 Linked Structures

See nodes, references and mutations.

- Singly linked list simulation
- Doubly linked lists
- Stacks and queues
- Hash tables

3 Trees and Heaps

Traverse hierarchical structures.

- Binary tree explorer
- BST operations
- DFS and BFS
- Heaps and priority queues

4 Graphs and Dynamic Programming

Model relationships and optimize repeated work.

- Graph builder
- BFS and DFS simulation
- Shortest paths
- Dynamic programming patterns

5 Problem Solving and Capstone

Integrate analysis, implementation and testing.

- Pattern selection
- Constraint-driven design
- Timed problem sets
- Algorithm visualizer capstone

Practice with purpose

Trace data structures, compare approaches and build a small visual explanation of an algorithm with test cases and complexity analysis.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

Visual algorithm explorer

Trace data structures, compare approaches and build a small visual explanation of an algorithm with test cases and complexity analysis.



Step-by-step graph or tree trace



Correctness and edge-case tests



Time and space complexity explanation



Tools and working methods

Interactive SVG simulations

Code editor and test runner

Complexity visualizer

Practice judge

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.



Delivery

Self-paced online · 12-16 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.



Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

- 1 Review the curriculum and your starting point.
- 2 Review current enrollment options and delivery details.
- 3 Start learning, keep your evidence and track your progress.

[Open the course experience →](#)

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Trace breadth-first search using a queue

A graph represents rooms connected by corridors. Every corridor has the same traversal cost. Find the fewest corridor steps from room A to each reachable room.

Understand the concept

Breadth-first search visits vertices in increasing distance from the start when every edge has equal weight. Use a FIFO queue and mark a vertex visited when you enqueue it, not after removing it. This prevents duplicate queue entries caused by multiple incoming edges. Maintain a parent map if you need to reconstruct the route.

Worked example

```
Edges: A-B, A-C, B-D, C-D, D-E
Start: queue=[A], distance[A]=0
Visit A: queue=[B,C], distance[B]=distance[C]=1
Visit B: queue=[C,D], distance[D]=2
Visit C: D is already discovered
Visit D: queue=[E], distance[E]=3
```

Interpret the result

One shortest route to E is A-B-D-E, with three edges. A-C-D-E is equally short. With adjacency lists, the traversal takes $O(V + E)$ time and $O(V)$ auxiliary space.

Knowledge check

Does ordinary BFS guarantee the lowest-cost path when edges have different weights?

Explanation: No. BFS minimizes edge count; varying edge costs require an algorithm suited to weighted shortest paths.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Trace breadth-first search using a queue



Your task

- Animate the queue, current vertex and discovered set separately.
- Reconstruct a shortest route from the parent map.
- Add a disconnected vertex and display an unreachable state.



Submission review criteria

- Each vertex enters the queue at most once
- Shortest edge-count route for an unweighted graph
- Correct handling of cycles, disconnected nodes and the start vertex

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Trace data structures, compare approaches and build a small visual explanation of an algorithm with test cases and complexity analysis.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

[Explore the worked lesson](#) →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.